



CI/CD — Complete Notes

Continuous Integration & Delivery/Deployment · Ship code automatically & safely

Practice **CI/CD**

Goal **Automate delivery**

CI **Build + Test**

CD **Release + Deploy**

A reader-friendly, all-in-one reference for CI/CD.

Table of Contents

1. What is CI/CD
2. CI vs CD vs CD
3. The Pipeline & Its Stages
4. Core Concepts
5. CI/CD Tools
6. GitHub Actions Example
7. Testing in the Pipeline
8. Deployment Strategies
9. Secrets & Pipeline Security
10. IaC & GitOps
11. Best Practices
12. Quick Mental Model
13. Common Interview Questions

1. What is CI/CD

CI/CD is the practice of **automating how code goes from a commit to production** — building, testing, and releasing it through a repeatable **pipeline** instead of manual steps. It catches bugs early, removes human error from releases, and lets teams ship small changes **frequently and safely**.

💡 **Mental model:** every push flows through an automated assembly line — **build** → **test** → **release** → **deploy**. If any stage fails, the change stops there, so broken code never reaches users.

Why it matters: faster feedback, fewer integration headaches, smaller/safer releases, easy rollbacks, and consistency (the pipeline does the same thing every time).

2. ⚖️ CI vs CD vs CD

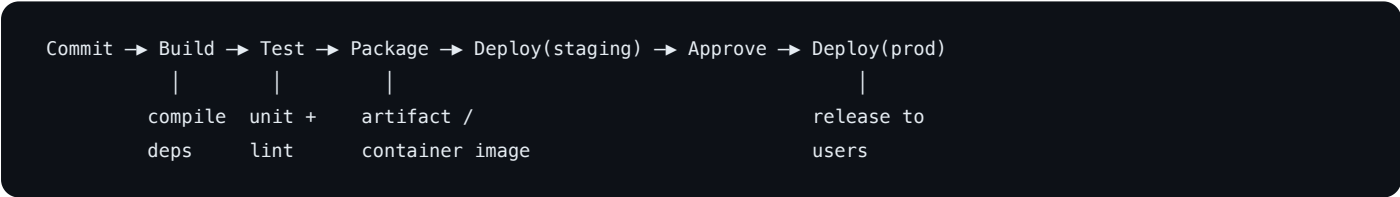
Three terms people blur together:

Term	Means	Stops at
Continuous Integration (CI)	Devs merge to a shared branch often; each merge auto- builds + tests	A validated build/artifact
Continuous Delivery (CD)	Every passing change is automatically prepared for release; deploy to prod is a manual approval/button	Ready-to-deploy, human clicks "go"
Continuous Deployment (CD)	Every passing change is automatically deployed to production — no human gate	Live in production

 The distinction that trips people up: **Continuous Delivery** = *automated up to* production (manual approval to release). **Continuous Deployment** = *automated all the way to* production (no manual step). CI is the build+test foundation both rely on.

3. 🏗️ The Pipeline & Its Stages

A **pipeline** is the automated sequence a change runs through. A typical flow:



Stage	What happens
Source	A push/PR/merge (or schedule) triggers the pipeline.
Build	Compile, install deps, produce an artifact (jar, binary, container image).
Test	Unit, integration, lint, security scans — fail fast.
Package/Publish	Push the artifact/image to a registry (ECR, GHCR, Artifactory).
Deploy	Release to staging, then (after gates) production.
Verify	Smoke tests, health checks, monitoring after deploy.

4. Core Concepts

Concept	Description
Pipeline / Workflow	The whole automated process definition (often YAML in the repo).
Stage / Job	A phase of the pipeline (build, test, deploy); jobs can run in parallel.
Step	A single command or action within a job.
Runner / Agent	The machine that executes jobs (hosted or self-hosted).
Artifact	The build output passed between stages / stored.
Trigger	What starts a pipeline (push, PR, tag, schedule, manual).
Environment	A deploy target (dev/staging/prod), often with protection rules.
Gate / Approval	A manual or automated check before promoting a change.
Cache	Reused dependencies to speed up builds.

5. CI/CD Tools

Tool	Notes
GitHub Actions	CI/CD built into GitHub; YAML workflows, huge marketplace.
GitLab CI/CD	Built into GitLab; <code>.gitlab-ci.yml</code> .
Jenkins	Veteran, self-hosted, plugin ecosystem; <code>Jenkinsfile</code> pipelines.
CircleCI / Travis CI	Popular hosted CI services.
AWS CodePipeline / CodeBuild / CodeDeploy	AWS-native pipeline, build, and deploy services.
Argo CD / Flux	GitOps CD for Kubernetes (sync cluster to Git).
Azure DevOps / Bitbucket Pipelines	Microsoft / Atlassian offerings.

They all share the same shape: a **YAML (or Jenkinsfile) pipeline** triggered by events, running **jobs on runners**, producing **artifacts**, and **deploying** to environments.

6. GitHub Actions Example

A minimal build → test → deploy workflow:

```
# .github/workflows/cicd.yml
name: CI/CD
on:
  push: { branches: [main] }
  pull_request:
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint && npm test
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: dist, path: dist/ }

  deploy:
    needs: build-test          # only after build-test passes
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production    # can require manual approval
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh
    env:
      AWS_ROLE: ${ secrets.AWS_DEPLOY_ROLE }
```

`needs:` chains jobs; `environment:` adds protection/approval; `${ secrets.* }` injects credentials securely.

7. Testing in the Pipeline

The "test pyramid" — fast, broad checks first; slow, narrow ones later:

- **Unit tests** — fast, run on every commit (the bulk).
- **Integration tests** — components together (DB, APIs).
- **End-to-end (E2E)** — full user flows (fewer, slower).
- **Static analysis** — lint, type-check, **SAST** (security), dependency/vuln scans, container image scans.
- **Fail fast** — order cheap checks first so failures surface in seconds, not minutes.

8. 🚀 Deployment Strategies

Strategy	How	Trade-off
Rolling	Replace instances gradually	Simple; mixed versions briefly
Blue/Green	Stand up a full new env, switch traffic over	Instant rollback; doubles resources during switch
Canary	Send a small % of traffic to the new version, then ramp	Limits blast radius; needs good monitoring
Recreate	Stop old, start new	Simplest; causes downtime
Feature flags	Ship code dark, toggle on later	Decouples deploy from release; flag management overhead

Pair any strategy with **health checks + automated rollback** so a bad release is reverted fast.

9. 🔒 Secrets & Pipeline Security

- **Never hard-code secrets** — use the platform's **secrets store** (GitHub Actions secrets, GitLab CI variables) or a manager (AWS Secrets Manager, Vault).
- Prefer **short-lived credentials** — e.g. **OIDC** federation so the pipeline assumes an **IAM role** instead of storing long-lived AWS keys.
- **Least privilege** for the pipeline's deploy identity; scope per environment.
- **Pin action/tool versions** (supply-chain safety), scan dependencies and images, and protect `main` with required checks.
- Restrict who can edit pipeline files and approve production deploys.

10. 📖 IaC & GitOps

- **Infrastructure as Code (IaC)** — provision infra (Terraform/CloudFormation) **through the pipeline**, so environments are reproducible and reviewed like app code.
- **GitOps** — Git is the **single source of truth** for both app and infra. A controller (**Argo CD / Flux**) continuously **syncs** the running environment to match the repo; you change infra by merging a PR, not by clicking in a console.

11. ✅ Best Practices

- **Commit small and often**; keep `main` always releasable.
- **Automate build + test on every PR**; gate merges on green checks.
- **Fail fast** — cheap checks (lint/unit) before slow ones (E2E).
- **One artifact, promote it** — build once, deploy the same artifact through envs.
- **Keep pipelines as code** in the repo and version them.
- **Cache dependencies** for speed; run independent jobs in parallel.
- **Use short-lived OIDC credentials**, least-privilege deploy roles, and a secrets store.

- **Safe deploys** — blue/green or canary + health checks + **automated rollback**.
 - **Make pipelines fast** (target < 10 min) so feedback stays tight.
-

12. 🧠 Quick Mental Model

- **CI/CD = automate commit → build → test → release → deploy.**
 - **CI** = merge often, auto build + test. **Continuous Delivery** = automated up to prod (manual approve). **Continuous Deployment** = automated all the way to prod.
 - A **pipeline** runs **jobs** on **runners**, producing **artifacts**, deploying to **environments** through **gates**.
 - **Test pyramid + fail fast; build once, promote the same artifact.**
 - Deploy safely with **blue/green / canary + health checks + rollback**.
 - Secure it: **secrets store + short-lived OIDC creds + least privilege + pinned versions**.
 - **IaC** provisions infra in the pipeline; **GitOps** syncs the environment to Git.
-

13. ? Common Interview Questions

Rapid-fire questions interviewers ask about CI/CD:

- **Q: What's the difference between CI and CD?** — CI = continuously integrate, build, and test merged code. CD = continuously deliver (auto up to prod, manual approval) or deploy (auto all the way to prod).
 - **Q: Continuous Delivery vs Continuous Deployment?** — Delivery has a manual approval before production; Deployment is fully automated to production with no human gate.
 - **Q: What are typical pipeline stages?** — Source/trigger → build → test → package/publish → deploy (staging→prod) → verify.
 - **Q: What is an artifact?** — The build output (binary, jar, container image) produced once and promoted through environments.
 - **Q: Blue/green vs canary?** — Blue/green switches all traffic to a parallel new environment (instant rollback). Canary shifts a small % first, then ramps (limits blast radius).
 - **Q: How do you handle secrets in a pipeline?** — Use the platform's secrets store or a secrets manager; prefer short-lived OIDC credentials over stored long-lived keys; least privilege.
 - **Q: How do you keep a pipeline fast?** — Fail fast (cheap checks first), cache dependencies, run jobs in parallel, build the artifact once.
 - **Q: What is GitOps?** — Git is the source of truth; a controller (Argo CD/Flux) continuously syncs the environment to match the repo.
 - **Q: How do you roll back a bad deploy?** — Automated rollback on failed health checks; with blue/green, switch traffic back to the old environment instantly.
 - **Q: Why build the artifact only once?** — To guarantee the exact thing you tested is what reaches production (no per-environment rebuild drift).
-

 Notes compiled as a quick reference for CI/CD.